

Grafo RDF de datos estadísticos

Víctor Tilve

2025-08-13

Comienza el código con las llamadas a las librerías necesarias, la ruta de la carpeta donde se encuentran los archivos para trabajar y una función para leer un grafo y extraer la información que hay en él mediante una consulta SPARQL.

```
library(readr)
library(ggplot2)
library(dplyr)

##
## Attaching package: 'dplyr'
##
## The following objects are masked from 'package:stats':
##
##   filter, lag
##
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
library(rdflib)

rdf_data <- "/media/victor/1FPB-COTARE/Cousas/TFM-IA/TFM-trabajo/00.CousasDefinitivas/clustering"

#Función para procesar un solo archivo RDF/XML
#Mucho ojo con los prefijos porque en ex el final / o # debe coincidir con el grafo
procesar_rdf <- function(file_path) {
  rdf <- rdf_parse(file_path, format = "rdfxml")
  consulta <- '
PREFIX sosa: <http://www.w3.org/ns/sosa/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX ex: <http://example.org#>

SELECT ?observacion ?sensor ?fecha ?valor ?noche ?luna ?estacion ?finSemana
WHERE {
  ?observacion rdf:type sosa:Observation ;
    sosa:madeBySensor ?sensor ;
    sosa:resultTime ?fecha ;
    sosa:hasSimpleResult ?valor ;
    ex:esNocheAstronomica ?noche ;
    ex:hayBrilloLuna ?luna ;
    ex:perteneceAEstacion ?estacion;
    ex:finDeSemana ?finSemana .
}'
}
```

```

resultado <- rdf_query(rdf, consulta)
df <- as.data.frame(resultado)

df$valor <- as.numeric(df$valor)
df$noche <- tolower(df$noche) == "true"
df$luna <- tolower(df$luna) == "true"
df$finSemana <- tolower(df$finSemana) == "true"
df$fecha <- as.POSIXct(df$fecha, format="%Y-%m-%dT%H:%M:%S", tz="UTC")

return(df)
}

```

A continuación se leen todos los archivos RDF de la carpeta indicada anteriormente y se combinan.

```

# Lista de archivos RDF en la carpeta
archivos <- list.files(rdf_data, pattern = "\\02.xml$", full.names = TRUE)

# Procesar y combinar todos los data.frames
todos_los_datos <- bind_rows(lapply(archivos, procesar_rdf))

```

En este paso se seleccionan los datos que interesan para realizar los cálculos estadísticos con ellos.

```

datos_filtrados <- subset(todos_los_datos, noche == TRUE & luna == FALSE)
rm(todos_los_datos)
datos_filtrados$solo_fecha <- as.Date(datos_filtrados$fecha) # solo la fecha (YYYY-MM-DD)
datos_filtrados$solo_hora <- format(datos_filtrados$fecha,
                                   format = "%H:%M:%S") # solo la hora
datos_filtrados <- subset(datos_filtrados, valor > 0)
#Valores atípicos

lim_inf <- quantile(datos_filtrados$valor, 0.01, na.rm = TRUE)
lim_sup <- quantile(datos_filtrados$valor, 0.99, na.rm = TRUE)
datos_filtrados <- subset(datos_filtrados, valor >= lim_inf & valor <= lim_sup)

#Valores atípicos

Q1 <- quantile(datos_filtrados$valor, 0.25, na.rm = TRUE)
Q3 <- quantile(datos_filtrados$valor, 0.75, na.rm = TRUE)
IQR_valor <- Q3 - Q1

lim_inf <- Q1 - 1.5 * IQR_valor
lim_sup <- Q3 + 1.5 * IQR_valor

outliers <- datos_filtrados %>%
  filter(valor < lim_inf | valor > lim_sup)

datos_filtrados <- subset(datos_filtrados, valor >= lim_inf & valor <= lim_sup)

#Etiqueta
datos_filtrados$etiqueta_sensor <- sub(".*#([_]+_[_]+).*", "\\1", datos_filtrados$sensor)

```

A continuación se definen las funciones necesarias para generar y serializar el grafo RDF que se quiere como resultado de este proceso. Dentro de esta misma función es donde se realizan los cálculos matemáticos necesarios para la obtención de los datos estadísticos.

```

library(rdfliib)

# -----
# generate_boxplot_rdf()
# - Filtra datos por sensor y por un intervalo temporal
# - Calcula los estadísticos de un boxplot (min, Q1, mediana, Q3, max)
# y además media, desviación típica y moda
# - Crea un grafo RDF con STATO + SOSA + PROV + TIME y lo serializa a Turtle
# -----
generate_boxplot_rdf <- function(g,df, ns,
                                # valor en la columna del sensor para filtrar
                                sensor_id,
                                # inicio del intervalo (carácter o POSIXct)
                                period_start,
                                # fin del intervalo
                                period_end,
                                # nombre del periodo
                                period_label,
                                # URI del sensor (opcional)
                                sensor_uri = NULL,
                                # etiqueta legible del sensor (opcional)
                                sensor_label = NULL,
                                # nombre de la columna con el id del sensor
                                sensor_col = "sensor",
                                # nombre de la columna con valores numéricos
                                value_col = "valor",
                                # nombre de la columna con las fechas
                                time_col = "fecha",
                                base_uri = "http://example.org#",
                                outfile = "grafo_estadistica.ttl",
                                # decimales para escribir los números
                                digits = 6,
                                # cómo estimar la moda: "density" o "hist"
                                mode_method = c("density","hist"),
                                # nº de bins si mode_method = "hist"
                                hist_breaks = 30)
{
  mode_method <- match.arg(mode_method)

  # --- Utilidades internas ---
  to_posix <- function(x) {
    # Normaliza a POSIXct en UTC. Si ya lo es, no hace nada.
    if (inherits(x, "POSIXt")) x else as.POSIXct(x, tz = "UTC")
  }
  fmt_id <- function(t) {
    # Convierte un instante a un identificador compacto: 20230101T000000Z
    format(t, "%Y%m%dT%H%M%SZ", tz = "UTC")
  }
  num <- function(x) {
    # Formatea números como literales decimales (sin notación científica)
    formatC(x, digits = digits, format = "fg")
  }
}

```

```

dtlex <- function(t) {
  # Datetime en XSD dateTime (ISO 8601, en UTC)
  format(t, "%Y-%m-%dT%H:%M:%SZ", tz = "UTC")
}

get_fragment <- function(uris) {
  uris <- gsub("<|>$", "", uris) # quita < > si vienen así
  ifelse(grepl("#", uris), sub("^.*#", "", uris), uris)
}

# --- Normaliza y valida el intervalo temporal ---
t_start <- to_posix(period_start)
t_end <- to_posix(period_end)
stopifnot(!is.na(t_start), !is.na(t_end), t_end > t_start)

# --- Comprobaciones de columnas requeridas ---
stopifnot(all(c(sensor_col, value_col, time_col) %in% names(df)))

# --- Filtrado por sensor y limpieza básica ---
sub <- df[df[[sensor_col]] == sensor_id &
  !is.na(df[[value_col]]) &
  !is.na(df[[time_col])], ]
# Asegura que la columna temporal sea POSIXct
if (!inherits(sub[[time_col]], "POSIXt")) sub[[time_col]] <- to_posix(
  sub[[time_col]])

# Filtrado por intervalo [t_start, t_end]
sub <- sub[sub[[time_col]] >= t_start & sub[[time_col]] <= t_end, ]

# Extrae el vector de valores numéricos y quita no finitos
v <- as.numeric(sub[[value_col]])
v <- v[is.finite(v)]
if (length(v) < 5) stop("Hay menos de 5 observaciones en el intervalo
  (se necesitan 5).")

# --- Cálculo de estadísticos ---
Q1 <- as.numeric(quantile(v, .25))
med <- as.numeric(quantile(v, .5))
Q3 <- as.numeric(quantile(v, .75))
minv <- min(v)
maxv <- max(v)
IQRv <- Q3 - Q1
meanv <- mean(v)
sdv <- sd(v)

# Moda (no es única en continuo; se estima):
if (mode_method == "density") {
  d <- density(v) # estimación por KDE
  modev <- d$x[which.max(d$y)]
} else {
  h <- hist(v, breaks = hist_breaks, plot = FALSE) # por histograma
  modev <- h$mids[which.max(h$counts)]
}

```

```

# --- Construcción de URIs "legibles" ---
period_id <- paste0(period_label, "_", fmt_id(t_start), "_", fmt_id(t_end))

dataset_uri <- paste0(base_uri, "dataset_", get_fragment(sensor_id), "_",
                      period_id) # colección SOSA
interval_uri <- paste0(base_uri, "intervalo_", period_id) # intervalo TIME
tbegin_uri <- paste0(base_uri, "tbegin_", period_id) # instante inicio
tend_uri <- paste0(base_uri, "tend_", period_id) # instante fin
calc_uri <- paste0(base_uri, "calc_", period_id) # instante cálculo (resultTime)
boxplot_uri <- paste0(base_uri, "boxplot_", get_fragment(sensor_id), "_", period_id) # el recurs

# Helper para crear URIs de cada estadístico
mk <- function(name) paste0(base_uri, name, "_", get_fragment(sensor_id), "_", period_id)
IQR_uri <- mk("IQR")
Q1_uri <- mk("Q1")
med_uri <- mk("median")
Q3_uri <- mk("Q3")
min_uri <- mk("min")
max_uri <- mk("max")
mean_uri <- mk("average")
sd_uri <- mk("stddev")
mode_uri <- mk("mode")

expand_curie <- function(term, ns) {
  stopifnot(is.character(term), length(term) == 1L)
  stopifnot(is.character(ns), !is.null(names(ns)))

  t <- trimws(term)

  # Si ya viene con <...>, quita los ángulos
  if (grepl("^<.*>$", t)) t <- sub("^<|>$", "", t)

  # Si ya es IRI completa (http/https) o blank node (_:...), devuelve tal cual
  if (grepl("^((https?|urn):)", t) || startsWith(t, "_:")) return(t)

  # Debe ser prefijo:local
  m <- regexpr(":", t, fixed = TRUE)
  if (m == -1) stop(sprintf("'s' no contiene prefijo (esperado prefijo:local').", term))

  prefix <- substr(t, 1, m - 1)
  local <- substr(t, m + 1, nchar(t))

  base <- ns[[prefix]]
  if (is.null(base))
    stop(sprintf("Prefijo '%s' no encontrado en 'ns'. Prefijos disponibles: %s",
                 prefix, paste(names(ns), collapse = ", ")))

  # Une base y local. La mayoría acaban en '/' o '#'; si no, añade '/'.
  if (!grepl("[/#]$", base) && !startsWith(local, "/")) {
    base <- paste0(base, "/")
  }
}

```

```

    paste0(base, local)
}

# --- Dataset (SOSA:ObservationCollection) con el tiempo del fenómeno ---
rdf_add(g, dataset_uri, expand_curie("rdf:type",ns), expand_curie("sosa:ObservationCollection",ns),
rdf_add(g, dataset_uri, expand_curie("sosa:madeBySensor",ns), sensor_uri,
    objectType = "uri")
rdf_add(g, dataset_uri, expand_curie("sosa:phenomenonTime",ns), interval_uri, objectType = "uri")
rdf_add(g, dataset_uri, expand_curie("sosa:observedProperty",ns), expand_curie("ex:NightSkyBrightness",ns),
    objectType = "uri")

# --- Intervalo temporal (TIME) ---
rdf_add(g, interval_uri, expand_curie("rdf:type",ns), expand_curie("time:Interval",ns), objectType = "uri")
rdf_add(g, interval_uri, expand_curie("time:hasBeginning",ns), tbegin_uri,
    objectType = "uri")
rdf_add(g, interval_uri, expand_curie("time:hasEnd",ns), tend_uri,
    objectType = "uri")

rdf_add(g, tbegin_uri, expand_curie("rdf:type",ns),
    expand_curie("time:Instant",ns), objectType = "uri")
rdf_add(g, tend_uri, expand_curie("rdf:type",ns),
    expand_curie("time:Instant",ns), objectType = "uri")
rdf_add(g, tbegin_uri, expand_curie("time:inXSDDateTime",ns),
    dtlex(t_start), datatype = "xsd:dateTime")
rdf_add(g, tend_uri, expand_curie("time:inXSDDateTime",ns),
    dtlex(t_end), datatype = "xsd:dateTime")

# --- resultTime (instante en que se calcularon los estadísticos) ---
now_utc <- dtlex(Sys.time())
rdf_add(g, calc_uri, expand_curie("rdf:type",ns),
    expand_curie("time:Instant",ns), objectType = "uri")
rdf_add(g, calc_uri, expand_curie("time:inXSDDateTime",ns),
    now_utc, datatype = "xsd:dateTime")

# --- boxplot (STATO:STATO_0000243) ---
rdf_add(g, boxplot_uri, expand_curie("rdf:type",ns), expand_curie("STATO:STATO_0000243",ns), objectType = "uri")
rdf_add(g, boxplot_uri, expand_curie("dctterms:subject",ns),
    sensor_uri, objectType = "uri")
rdf_add(g, boxplot_uri, expand_curie("sosa:phenomenonTime",ns),
    interval_uri, objectType = "uri") # sobre qué periodo
rdf_add(g, boxplot_uri, expand_curie("sosa:resultTime",ns),
    calc_uri, objectType = "uri") # cuándo se calculó
rdf_add(g, boxplot_uri, expand_curie("prov:wasDerivedFrom",ns),
    dataset_uri, objectType = "uri") # de qué datos parte

# Declara que el boxplot "tiene partes": cada estadístico como recurso propio
parts <- c(IQR_uri, Q1_uri, med_uri, Q3_uri, min_uri, max_uri)
for (p in parts) {
    rdf_add(g, boxplot_uri, expand_curie("dctterms:hasPart",ns), p,
        objectType = "uri")
}

# --- Helper para añadir cada parte con su tipo STATO, valor y procedencia ---
add_stat <- function(uri, tipo, value) {

```

```

rdf_add(g, uri, expand_curie("rdf:type",ns), tipo,
        objectType = "uri") # tipa el recurso
rdf_add(g, uri, expand_curie("rdf:value",ns), as.character(value),
        objectType = "literal",
        datatype = expand_curie("xsd:decimal", ns)) # el número
rdf_add(g, uri, expand_curie("sosa:phenomenonTime",ns), interval_uri,
        objectType = "uri") # periodo al que se refiere
rdf_add(g, uri, expand_curie("prov:wasDerivedFrom",ns), dataset_uri,
        objectType = "uri") # origen de datos
}

# --- Añade las partes con sus valores ---
add_stat(IQR_uri, expand_curie("STATO:STATO_0000164",ns), IQRv) # IQR
add_stat(Q1_uri, expand_curie("STATO:STATO_0000167",ns), Q1) # primer cuartil
add_stat(med_uri, expand_curie("STATO:STATO_0000574",ns), med) # mediana
add_stat(Q3_uri, expand_curie("STATO:STATO_0000170",ns), Q3) # tercer cuartil
add_stat(min_uri, expand_curie("STATO:STATO_0000150",ns), minv) # mínimo
add_stat(max_uri, expand_curie("STATO:STATO_0000151",ns), maxv) # máximo
add_stat(mean_uri, expand_curie("STATO:STATO_0000573",ns), meanv) # media
add_stat(sd_uri, expand_curie("STATO:STATO_0000237",ns), sdv) # desv típica
add_stat(mode_uri, expand_curie("STATO:STATO_0000033",ns), modev) # moda (estimada)

# --- Cálculo moda ----
if(mode_method=="density"){
  rdf_add(g,mode_uri, expand_curie("rdfs:comment",ns) ,"moda calculada para una variable continua con
}else if(mode_method=="hist"){
  rdf_add(g,mode_uri, expand_curie("rdfs:comment",ns) ,"moda calculada para una variable continua por
}
}

```

En el código que se ve a continuación se definen las funciones necesarias para recorrer un archivo yaml y leer sus datos para generar usando los datos cargados anteriormente los datos estadísticos para los periodos indicados en el archivo yaml. En la función aquí definida ya existen algunos intervalos de tiempo predefinidos (trimestres, cuatrimestres, etc.); pero se puede indicar fechas de inicio y fin de manera explícita para generar otros distintos.

```

# Necesario para leer YAML
library(yaml)

# Operador "o si no"
`%|||%` <- function(a, b) if (!is.null(a)) a else b
get_or <- function(lst, name, default) { val <- lst[[name]];
if (is.null(val)) default else val }

# Ejecuta generate_boxplot_rdf() para cada entrada del YAML y escribe un único TTL
run_stats_from_yaml <- function(df, yaml_path) {
  cfg <- yaml::read_yaml(yaml_path)

  # --- Defaults globales (opcionales) en config ---
  cfg_global <- cfg$config %|||% list()
  base_uri <- get_or(cfg_global, "base_uri", "http://example.org#")
  outfile <- get_or(cfg_global, "outfile", "grafo_estadistica.ttl")
  mode_method <- get_or(cfg_global, "mode_method", "density") # "density" o "hist"
}

```

```

hist_breaks <- get_or(cfg_global, "hist_breaks", 30)
digits      <- get_or(cfg_global, "digits",      6)

if (is.null(cfg$entries) || !length(cfg$entries))
  stop("El YAML no contiene 'entries'.")

# --- Un único grafo y prefijos para TODA la ejecución ---
g <- rdf()
ns <- c(
  ex    = "http://example.org#",
  STATO = "http://purl.obolibrary.org/obo/",
  dterms = "http://purl.org/dc/terms/",
  time   = "http://www.w3.org/2006/time#",
  prov    = "http://www.w3.org/ns/prov#",
  sosa    = "http://www.w3.org/ns/sosa/",
  rdfs    = "http://www.w3.org/2000/01/rdf-schema#",
  rdf     = "http://www.w3.org/1999/02/22-rdf-syntax-ns#",
  xsd     = "http://www.w3.org/2001/XMLSchema#"
)

# --- Recorre entradas y añade triples al mismo grafo ---
for (i in seq_along(cfg$entries)) {
  e <- cfg$entries[[i]]

  # Requeridos
  sensor_id    <- e$sensor_id    %||% stop(sprintf("Falta sensor_id
                                                    en entrada %d", i))
  period_start <- e$period_start %||% stop(sprintf("Falta period_start
                                                    en entrada %d", i))
  period_end   <- e$period_end   %||% stop(sprintf("Falta period_end
                                                    en entrada %d", i))

  # Opcionales
  period_label <- e$period_label %||% e$period_name %||% "periodo"
  sensor_uri   <- e$sensor_uri   %||% sensor_id
  sensor_label <- e$sensor_label %||% sub("^.*#", "", sensor_id)

  sensor_col   <- e$sensor_col   %||% "sensor"
  value_col    <- e$value_col    %||% "valor"
  time_col     <- e$time_col     %||% "fecha"

  mode_m_i     <- e$mode_method  %||% mode_method
  hist_b_i     <- e$hist_breaks  %||% hist_breaks
  digits_i     <- e$digits       %||% digits

  message(sprintf("(%d/%d) %s [%s-%s] -> %s",
                  i, length(cfg$entries), sensor_id, period_start, period_end, outfile))

  # IMPORTANTE: no re-crear g dentro; acumular en el MISMO grafo
  g <- generate_boxplot_rdf(
    g = g,
    ns = ns,
    df = df,

```



```

    sensor_id      = sensor_id,
    period_start   = period_start,
    period_end     = period_end,
    period_label   = period_label,
    sensor_uri     = sensor_uri,
    sensor_label   = sensor_label,
    sensor_col     = sensor_col,
    value_col      = value_col,
    time_col       = time_col,
    base_uri       = base_uri,
    outfile        = NULL,      # NO escribir aquí; se escribe al final
    digits         = digits_i,
    mode_method    = mode_m_i,
    hist_breaks    = hist_b_i
  )
}

# --- Serializa UNA sola vez al final ---
rdf_serialize(
  g, outfile,
  format      = "turtle",
  namespace   = ns,
  prefix      = names(ns)#,
  #base       = base_uri
)
message("Turtle escrito en: ", normalizePath(outfile, mustWork = FALSE))

invisible(g)
}

```

Ejemplo de uso de este código usando un archivo de configuración yaml. La variable df se obtiene ejecutando manualmente el código del comienzo de este informe.

```

df <- datos_filtrados
yaml_path <- "/media/victor/1FPB-COTARE/Cousas/TFM-IA/TFM-trabajo/00.CousasDefinitivas/clustering/boxp
run_stats_from_yaml(df, yaml_path)

```

```
## (1/2) http://example.org#SQM_Cervantes_ANC_42.820778_-6.922902 [2024-01-01 00:00:00-2024-03-31 23:59:59] -> grafo_estadistica.ttl
```

```
## (2/2) http://example.org#SQM_TorreHercules_COR_43.382763_-8.409202 [2024-01-01 00:00:00-2024-03-31 23:59:59] -> grafo_estadistica.ttl
```

```
## Turtle escrito en: /media/victor/1FPB-COTARE/Cousas/TFM-IA/TFM-trabajo/00.CousasDefinitivas/clustering/boxp
```